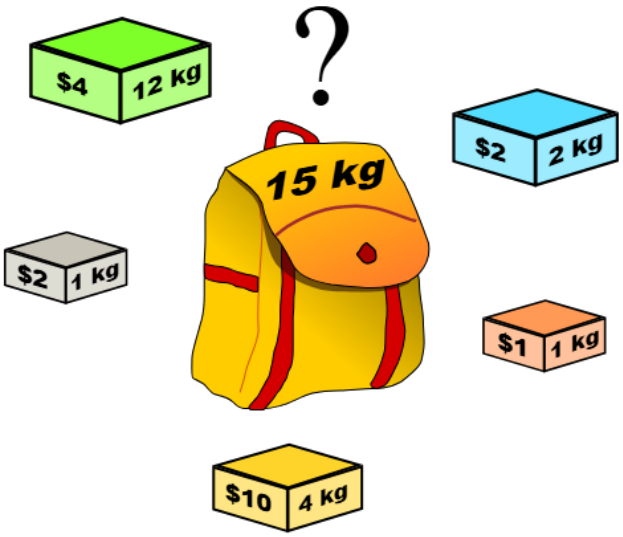
Tugas Besar 1

Penerapan Algoritma *Local Search* untuk Optimasi Pemilihan dan Penataan Paket dalam Peti Kemas

IF3170 Intelegensi Artifisial

Semester 1 Tahun 2026/2027



Disusun oleh:

Three Agents

Muhammad Aufar Rizqi Kusuma	(13524061)
Arina Azka	(13524049)
Michael James Liman	(13524106)

Laboratorium Intelegensi Buatan
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1 Pendahuluan **2**
1.1 *Backstory* . . . 2
1.2 *Overview* Permasalahan . . . 2
2 Landasan Teori **4**
2.1 Algoritma Hill Climbing . . . 4
2.1.1 *Steepest Ascent Hill Climbing* . . . 4
2.1.2 *Hill Climbing with Sideways Move* . . . 4
2.1.3 *Stochastic Hill Climbing* . . . 4
2.1.4 *Random Restart Hill Climbing* . . . 5
2.2 Algoritma Simulated Annealing . . . 5
2.3 Genetic Algorithm . . . 6
3 Implementasi Algoritma **8**
3.1 Representasi State . . . 8
3.2 *Constraint* Permasalahan . . . 8
3.3 *Succesor Function* . . . 8
3.4 *Objective Function* . . . 8
3.5 Formulasi *Initial State* . . . 8
3.6 Implementasi Hill Climbing . . . 8
3.6.1 Pembangkitan *Neighbor* dan Heuristik . . . 8
3.6.2 Parameter dan Kondisi Berhenti . . . 8
3.6.3 Metrik yang Dicatat . . . 9
3.6.4 *Source Code* . . . 9
3.6.5 Pelaksana Eksperimen dan Visualisasi . . . 12
3.7 Implementasi Simullated Annealing . . . 12
3.8 Implementasi Genetic Algorithm . . . 12
4 Hasil Eksperimen dan Analisis **13**
4.1 Metodologi Eksperimen . . . 13
4.2 Hasil Eksperimen dan Analisis tiap Algoritma . . . 13
4.2.1 Hill Climbing . . . 13
4.3 Hasil Eksperimen dan Analisis tiap Algoritma . . . 13
4.3.1 Hill Climbing . . . 13
4.3.2 Simulated Annealing . . . 13
4.3.3 Genetic Algorithm . . . 13
4.4 Analisis Keseluruhan . . . 13
5 Kesimpulan dan Saran **14**
5.1 Kesimpulan . . . 14
5.2 Saran . . . 14

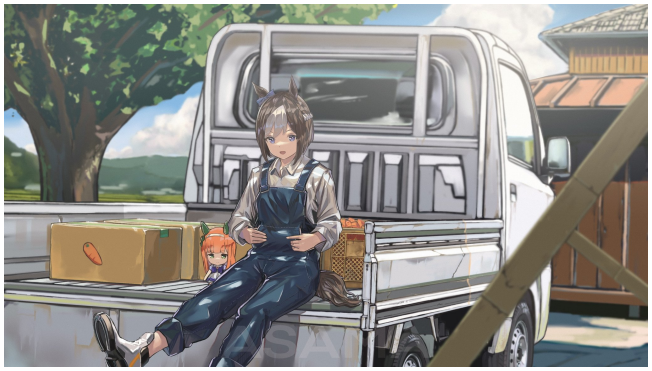
Bab 1

Pendahuluan

1.1 *Backstory*

Keluarga Mejiro sedang berencana mengekspansi bisnisnya ke bidang logistik darat dan ingin melakukan uji coba berskala kecil terlebih dahulu untuk torasen. Sebagai teman yang baik, Mejiro McQueen menawarkan posisi driver truk uji coba kepada Special Week, karena sudah sering menyetir truk di kampung halamannya di Hokkaido. Usaha baru ini bernama Mejiro Express dengan slogan "Lebih kilat dari Umamusume".

Karena Special Week sibuk berlatih untuk memenangkan lomba-lombanya, dia tidak bisa membuang waktu berlama-lama untuk menentukan pengaturan pengiriman yang optimal. Oleh karena itu, dia meminta kalian, toreena-toreena yang pintar dalam keinformatikawan, untuk membuat suatu program yang cerdas untuk menolongnya.



Gambar 1.1: Special Week, driver truk handal (Sumber: Manatsu no Gin Hourui Uma C108 Doyoubi Higashi Chiku Ho-10a on X)

1.2 *Overview* Permasalahan

Tugas Besar 1 mata kuliah IF3170 Inteligensi Artifisial menugaskan mahasiswa untuk mencari konfigurasi pemuatan paket yang optimal pada sebuah truk peti kemas. Secara umum, paket memiliki ukuran, berat, dan nilai yang beragam, sedangkan ruang serta kapasitas angkut truk terbatas. Tujuan optimasi adalah memilih dan menata paket agar nilai total paket yang berhasil dimuat menjadi sebesar mungkin.

Permasalahan ini tidak hanya berkaitan dengan pemilihan paket bernilai tinggi, tetapi juga dengan kemungkinan penataannya di dalam ruang tiga dimensi. Susunan paket harus tetap layak untuk diangkut dengan memperhatikan keterbatasan ruang, kestabilan tumpukan, kondisi barang pecah-belah, dan kapasitas truk. Beragamnya kombinasi paket dan penataannya membuat pencarian konfigurasi terbaik menjadi sulit dilakukan secara menyeluruh.

Untuk mencari solusi, tugas ini meminta implementasi tiga algoritma pencarian, yaitu satu varian Hill Climbing,

Simulated Annealing, dan Genetic Algorithm. Algoritma-algoritma tersebut mengeksplorasi konfigurasi secara bertahap dari solusi awal yang dibentuk secara acak, lalu berusaha menemukan konfigurasi dengan nilai objektif yang lebih baik. Kinerja pencarian kemudian dibandingkan melalui kualitas solusi, perubahan nilai objektif selama proses, durasi, dan konsistensi hasil eksperimen.

Bab 2

Landasan Teori

2.1 Algoritma Hill Climbing

Hill climbing adalah algoritma *local search* yang bekerja pada satu *state* saja. Algoritma ini mulai dari sebuah *state* awal, membangkitkan *neighbor* dari *state* saat ini, lalu berpindah ke *neighbor* yang nilai *objective function*-nya lebih baik. Pencarian berhenti ketika tidak ada lagi *neighbor* yang lebih baik. Karena hanya menyimpan satu *state* dan tidak menyimpan jalur pencarian, kebutuhan memorinya sangat kecil dan setiap iterasinya murah. Pada permasalahan ini *objective function* dimaksimalkan, sehingga algoritma bergerak "naik" menuju nilai *value* total paket yang lebih besar.

Kelemahan utama *hill climbing* adalah tidak adanya mekanisme untuk keluar dari daerah yang buruk. Ada tiga kondisi yang menyebabkan pencarian terhenti sebelum mencapai *global optimum*.

- *Local maximum*: *state* yang lebih baik dari semua *neighbor*-nya, tetapi bukan yang terbaik secara global.
- *Plateau*: daerah datar tempat semua *neighbor* bernilai sama, sehingga tidak ada arah yang jelas untuk naik.
- *Shoulder*: *plateau* yang di ujungnya masih terdapat jalan menaik, sehingga pencarian yang berhenti terlalu cepat melewatkannya.

Keempat varian berikut dibuat untuk mengurangi kelemahan tersebut dengan cara yang berbeda.

2.1.1 *Steepest Ascent Hill Climbing*

Pada setiap iterasi, algoritma mengevaluasi sekumpulan *neighbor* dan memilih *neighbor* dengan nilai terbaik. Algoritma berpindah ke *neighbor* tersebut hanya jika nilainya *strictly* lebih besar daripada nilai *state* saat ini; jika tidak, algoritma berhenti karena telah mencapai *local optimum*. Varian ini deterministik terhadap pilihan langkahnya dan berhenti paling cepat, tetapi paling mudah terjebak pada *local maximum* dan *plateau*.

2.1.2 *Hill Climbing with Sideways Move*

Varian ini sama dengan *steepest ascent*, tetapi juga mengizinkan perpindahan ke *neighbor* yang nilainya sama dengan *state* saat ini (*sideways move*). Dengan cara ini algoritma dapat menyeberangi *plateau* dan *shoulder*. Agar tidak berputar tanpa akhir pada *plateau* yang tidak memiliki jalan keluar, jumlah *sideways move* dibatasi oleh parameter *maximum sideways move*. Pencarian dihentikan ketika jumlah *sideways move* mencapai batas tersebut.

2.1.3 *Stochastic Hill Climbing*

Algoritma memilih satu *neighbor* secara acak pada setiap iterasi dan berpindah ke *neighbor* tersebut hanya jika nilainya lebih baik. Varian ini tidak perlu mengevaluasi seluruh *neighbor* dalam satu iterasi sehingga setiap iterasinya lebih murah, dan sifat acaknya membuat jalur pencarian lebih bervariasi. Karena tidak ada kondisi

"tidak ada *neighbor* yang lebih baik" yang dapat dipastikan dari satu sampel, pencarian dihentikan setelah jumlah iterasi maksimum tercapai.

2.1.4 *Random Restart Hill Climbing*

Algoritma menjalankan *steepest ascent hill climbing* berulang kali, setiap kali dimulai dari *state* awal acak yang baru, dan menyimpan hasil terbaik dari seluruh percobaan. Peluang menemukan *global optimum* meningkat seiring bertambahnya jumlah *restart*, karena setiap *restart* mengeksplorasi daerah ruang pencarian yang berbeda. Pencarian dihentikan ketika jumlah *restart* mencapai *maximum restart*. Kelemahannya adalah biaya komputasi yang bertambah sebanding dengan jumlah *restart*.

2.2 Algoritma Simulated Annealing

Simulated Annealing adalah algoritma pencarian lokal stokastik yang meniru proses pendinginan material untuk mencari solusi bernilai objektif tinggi. Berbeda dari Hill Climbing yang hanya menerima perpindahan ke solusi lebih baik, Simulated Annealing juga dapat menerima solusi yang lebih buruk dengan peluang tertentu. Mekanisme ini membantu pencarian keluar dari optimum lokal dan mengeksplorasi bagian lain dari ruang solusi [1, 2].

Pada setiap iterasi, algoritma memilih satu solusi tetangga dan menghitung perubahan nilai objektifnya. Untuk persoalan maksimisasi, perubahan tersebut didefinisikan sebagai $\Delta E = f(s') - f(s)$, dengan s sebagai solusi saat ini dan s' sebagai kandidat tetangga. Kandidat yang meningkatkan atau mempertahankan nilai objektif selalu diterima. Kandidat yang menurunkan nilai objektif dapat diterima dengan peluang yang bergantung pada temperatur T, seperti dirumuskan pada Persamaan 2.1 [2].

$$P(\text{menerima } s') = \begin{cases} 1, & \Delta E \geq 0, \\ \exp\left(\dfrac{\Delta E}{T}\right), & \Delta E < 0. \end{cases} \tag{2.1}$$

Temperatur mengendalikan tingkat eksplorasi algoritma. Pada temperatur tinggi, penurunan nilai objektif masih mungkin diterima sehingga pencarian dapat menjelajahi ruang solusi dengan lebih luas. Ketika temperatur diturunkan, peluang menerima perpindahan yang lebih buruk ikut mengecil dan pencarian semakin berfokus pada perbaikan. Salah satu jadwal pendinginan yang umum digunakan adalah pendinginan geometris, dengan $0 < \alpha < 1$ sebagai faktor pendinginan [1].

$$T_{k+1} = \alpha T_k, \qquad 0 < \alpha < 1. \tag{2.2}$$

Keunggulan Simulated Annealing adalah kemampuannya melewati optimum lokal dengan menerima perpindahan yang sesaat menurunkan nilai objektif, tanpa harus menyimpan banyak kandidat solusi. Namun, kualitas hasil dan waktu pencarian sensitif terhadap temperatur awal, faktor pendinginan, temperatur minimum, serta mekanisme pembentukan tetangga. Jika temperatur turun terlalu cepat, pencarian masih dapat terjebak pada optimum lokal; frekuensi kondisi ini menjadi salah satu metrik evaluasi algoritma. Karena itu, algoritma ini tidak menjamin solusi global terbaik untuk jadwal pendinginan berdurasi terbatas dan hasilnya dapat berbeda antarpercobaan [1, 2].

Simulated Annealing cocok digunakan pada ruang solusi besar dengan banyak optimum lokal, terutama ketika evaluasi solusi cukup murah dan solusi mendekati optimal sudah bermanfaat. Dalam persoalan pemuatan paket, pencarian dimulai dari konfigurasi acak. Pada setiap iterasi, satu tetangga dibentuk melalui operasi pertukaran paket, pemindahan paket, atau rotasi. Algoritma menerima kandidat yang meningkatkan nilai objektif, dan sesekali menerima kandidat yang menurunkannya sesuai peluang pada Persamaan 2.1. Temperatur kemudian dikurangi berdasarkan jadwal pendinginan. Nilai $\exp(\Delta E/T)$ menjadi ukuran peluang penerimaan kandidat yang lebih buruk, sesuai metrik yang diminta spesifikasi untuk dianalisis terhadap iterasi.

Pseudocode berikut menggunakan pendinginan geometris dan berhenti ketika temperatur mencapai nilai minimum. Variabel s_{best} menyimpan konfigurasi bernilai tertinggi yang ditemukan, meskipun proses menerima perpindahan sementara ke konfigurasi yang lebih buruk.

Algorithm 1: Simulated Annealing

Data: Solusi awal s_0, fungsi tetangga N, fungsi objektif f, parameter $T_0 > T_{\min} > 0$ dan $0 < \alpha < 1$
Result: Solusi terbaik yang ditemukan s_{best}

```
s ← s_0, s_best ← s_0, T ← T_0
while T > T_min do
    if N(s) = ∅ then
        return s_best
    end if
    Pilih s' secara acak dari N(s)
    ΔE ← f(s') − f(s)
    if ΔE ≥ 0 then
        s ← s'
    else
        Bangkitkan u secara acak dari [0, 1]
        if u < exp(ΔE/T) then
            s ← s'
        end if
    end if
    if f(s) > f(s_best) then
        s_best ← s
    end if
    T ← αT
end while
return s_best
```

2.3 Genetic Algorithm

Genetic Algorithm adalah metode pencarian berbasis populasi yang memperbarui kumpulan kandidat solusi melalui mekanisme yang terinspirasi oleh seleksi alam. Setiap kandidat disebut individu, sedangkan nilai objektifnya berperan sebagai ukuran fitness. Pada tiap generasi, algoritma memilih individu sebagai induk, menggabungkan bagian solusi melalui crossover, lalu menerapkan mutasi secara acak untuk membentuk populasi baru [1, 3].

Karakteristik Genetic Algorithm adalah pencarian atas banyak kandidat secara bersamaan dan penggunaan operator stokastik untuk menghasilkan generasi berikutnya. Seleksi roulette memberi peluang lebih besar kepada individu dengan fitness lebih tinggi. Jika fitness individu ke-i adalah f_i dan bobot seleksinya w_i, peluang memilih individu tersebut dirumuskan sebagai

$$P(i) = \frac{w_i}{\sum_{j=1}^{n} w_j}. \tag{2.3}$$

Untuk persoalan pemuatan paket dengan nilai objektif nonnegatif, bobot seleksi dapat menggunakan nilai objektif. Jika seluruh bobot bernilai nol, seleksi dilakukan secara seragam. Crossover satu titik menggabungkan gen dari dua induk, sementara mutasi mengubah satu gen dengan peluang tertentu. Kedua operator harus menghasilkan individu yang memenuhi batasan pemuatan paket [3].

Keunggulan Genetic Algorithm adalah kemampuannya mengeksplorasi beberapa wilayah ruang solusi secara paralel dan mengurangi ketergantungan pencarian pada satu konfigurasi awal. Keragaman populasi dapat membantu pencarian menghindari optimum lokal. Namun, hasil dan waktu pencarian bergantung pada ukuran populasi, jumlah generasi, operator, serta parameter seperti laju mutasi. Algoritma ini tidak menjamin solusi global terbaik dalam jumlah generasi terbatas, dan setiap generasi memerlukan evaluasi banyak kandidat [1, 3].

Genetic Algorithm sesuai untuk ruang solusi besar dengan representasi kandidat yang dapat digabungkan dan dimutasi, khususnya ketika solusi layak dapat dipulihkan setelah operator diterapkan. Pada persoalan pemuatan paket, satu individu merepresentasikan penempatan seluruh paket. Populasi awal memuat konfigurasi awal yang valid dan konfigurasi valid lain yang dibentuk secara acak. Di setiap generasi, individu dievaluasi dengan fungsi objektif, induk dipilih menggunakan roulette, kemudian crossover satu titik dan mutasi dilakukan untuk

membentuk populasi berikutnya. Individu terbaik disimpan terpisah sepanjang pencarian agar solusi terbaik tetap tersedia meskipun tidak diwariskan ke generasi baru.

Pseudocode berikut menggunakan seleksi roulette, crossover satu titik dengan perbaikan kelayakan, dan mutasi satu gen dengan peluang μ. Pencarian berhenti setelah jumlah generasi yang ditentukan dan mengembalikan individu terbaik yang pernah dievaluasi.

Algorithm 2: Genetic Algorithm

Data: Solusi awal valid s_0, fungsi objektif f, ukuran populasi n, jumlah generasi G, laju mutasi μ
Result: Individu terbaik yang ditemukan s_{best}

```
P ← populasi valid berukuran n yang memuat s_0
s_best ← s_0, f_best ← −∞
for g ← 0 sampai G do
    Evaluasi f(x) untuk setiap x ∈ P
    x_best ← arg max_{x∈P} f(x)
    if f(x_best) > f_best then
        s_best ← x_best, f_best ← f(x_best)
    end if
    if g < G then
        P_baru ← ∅
        while |P_baru| < n do
            Pilih induk p_1 dan p_2 dari P dengan seleksi roulette
            c ← crossover satu titik (p_1, p_2) dan perbaiki agar valid
            Dengan peluang μ, mutasikan satu gen pada c dan pertahankan validitasnya
            Tambahkan c ke P_baru
        end while
        P ← P_baru
    end if
end for
return s_best
```

Bab 3

Implementasi Algoritma

3.1 Representasi State

3.2 *Constraint* Permasalahan

3.3 *Succesor Function*

3.4 *Objective Function*

3.5 Formulasi *Initial State*

3.6 Implementasi Hill Climbing

Seluruh varian *hill climbing* diimplementasikan pada berkas `src/algorithms/hill_climbing/algorithm.py` di atas representasi *state*, validator *constraint*, dan fungsi *neighbor* yang sama dengan algoritma lainnya (`State`, `validate_state`, dan `random_neighbor`). Dengan demikian seluruh algoritma menerima masukan dan mengembalikan keluaran yang seragam, yaitu objek `SearchResult` yang memuat *state* awal, *state* akhir, nilai *objective function* akhir, riwayat nilai per iterasi, durasi pencarian, dan metrik khusus algoritma.

3.6.1 Pembangkitan *Neighbor* dan Heuristik

Ruang *neighbor* pada permasalahan ini sangat besar karena koordinat paket berupa bilangan bulat, sehingga seluruh *neighbor* tidak dapat dienumerasi pada setiap iterasi. Sebagai gantinya, setiap *neighbor* dibangkitkan oleh `random_neighbor`, yang secara acak memilih salah satu dari tiga operasi (menukar dua paket, memindahkan paket ke koordinat lain, atau merotasi paket 90°) dan hanya mengembalikan *state* yang lolos validasi *constraint*.

Heuristik yang kami gunakan adalah *steepest* berbasis sampel: pada setiap iterasi dibangkitkan `neighbors_per_iteration` (bawaan 30) *neighbor* valid, kemudian *neighbor* dengan nilai terbaik dianggap sebagai *neighbor* terbaik. Konsekuensinya, varian *steepest* dan *sideways* tidak menjamin menemukan *neighbor* terbaik yang sesungguhnya, tetapi peluangnya naik seiring bertambahnya ukuran sampel. Varian *stochastic* memakai tepat satu *neighbor* per iterasi.

3.6.2 Parameter dan Kondisi Berhenti

- *Steepest*: berhenti ketika *neighbor* terbaik dari sampel tidak lebih baik daripada *state* saat ini.
- *Sideways*: sama seperti *steepest*, tetapi *neighbor* bernilai sama diterima selama jumlah *sideways move* belum mencapai `max_sideways`.
- *Stochastic*: berhenti setelah `max_iterations` iterasi. Iterasi yang *neighbor*-nya ditolak tetap dihitung.
- *Random restart*: menjalankan *steepest* dari *state* awal yang diberikan, lalu dari `max_restarts` *state* acak

Parameter	Bawaan	Keterangan
`variant`	`steepest`	`steepest`, `sideways`, `stochastic`, atau `random_restart`
`neighbors_per_iteration`	30	Jumlah *neighbor* yang disampel per iterasi (tidak dipakai *stochastic*)
`max_iterations`	1000	Batas iterasi; menjadi kondisi berhenti pada *stochastic* dan batas pengaman pada varian lain
`max_sideways`	100	Batas *sideways move* (varian *sideways*)
`max_restarts`	10	Jumlah *restart* setelah percobaan pertama (varian *random restart*)
`seed`	—	Seed pembangkit acak agar eksperimen dapat diulang

Tabel 3.1: Parameter algoritma *hill climbing*

baru, sehingga total terdapat `max_restarts` + 1 pendakian. Hasil terbaik dari seluruh pendakian dikembalikan.

3.6.3 Metrik yang Dicatat

Setiap *run* mengembalikan `HillClimbingMetrics` yang berisi varian yang dipakai, banyak iterasi hingga berhenti, jumlah dan batas *sideways move*, jumlah dan batas *restart*, serta banyak iterasi pada setiap *restart*. Riwayat nilai *objective function* dicatat pada setiap iterasi (elemen pertama adalah nilai *state* awal) dan digunakan untuk membuat plot nilai *objective function* terhadap banyak iterasi. Durasi pencarian diukur dengan `perf_counter`.

3.6.4 *Source Code*

Fungsi `_climb` menjalankan satu pendakian untuk varian *steepest*, *sideways*, atau *stochastic*. Fungsi `hill_climbing` memvalidasi parameter, memilih varian, menangani pengulangan pada *random restart*, lalu menyusun `SearchResult`.

Listing 3.1: Fungsi bantu dan `_climb` pada `algorithm.py`

```
from math import isfinite
from random import Random
from time import perf_counter

from src.contracts import ObjectiveFunction, resolve_objective
from src.model import State
from src.neighbors import random_initial_state, random_neighbor
from src.results import HillClimbingMetrics, ObjectiveValue, SearchResult

VARIANTS = ("steepest", "sideways", "stochastic", "random_restart")


def _check_int(value, name: str, minimum: int) -> None:
    if type(value) is not int or value < minimum:
        raise ValueError(f"{name} must be an integer >= {minimum}")


def _score(objective: ObjectiveFunction[State], state: State) -> ObjectiveValue:
    score = objective(state)
    if isinstance(score, bool) or not isinstance(score, (int, float)) or not isfinite(score):
        raise ValueError("objective_fn must return finite integer or float values")
    return score


def _climb(
    start: State,
    objective: ObjectiveFunction[State],
    rng: Random,
    *,
    variant: str,
```

```
    neighbors_per_iteration: int,
    max_iterations: int,
    max_sideways: int,
) -> tuple[State, ObjectiveValue, list[ObjectiveValue], int, int]:
    """One␣climb.␣Returns␣(state,␣score,␣history,␣iterations,␣sideways_moves).

␣␣␣␣history[0]␣is␣the␣start␣score;␣one␣entry␣is␣appended␣per␣iteration.
␣␣␣␣"""
    current, current_score = start, _score(objective, start)
    history = [current_score]
    iterations = sideways = 0

    while iterations < max_iterations:
        if variant == "stochastic":
            candidates = [random_neighbor(current, rng)]
        else:
            candidates = [random_neighbor(current, rng) for _ in range(neighbors_per_iteration)]
        scored = [(_score(objective, c), c) for c in candidates]
        best_score, best = max(scored, key=lambda pair: pair[0])

        if best_score > current_score:
            current, current_score = best, best_score
        elif variant == "sideways" and best_score == current_score and sideways < max_sideways:
            current = best
            sideways += 1
        elif variant == "stochastic":
            iterations += 1  # a rejected random neighbor still costs an iteration
            history.append(current_score)
            continue
        else:
            break  # local optimum (or sideways budget exhausted)
        iterations += 1
        history.append(current_score)

    return current, current_score, history, iterations, sideways
```

Listing 3.2: Fungsi `hill_climbing` pada `algorithm.py`

```
def hill_climbing(
    initial_state: State,
    *,
    seed: int,
    objective_fn: ObjectiveFunction[State] | None = None,
    variant: str = "steepest",
    neighbors_per_iteration: int = 30,
    max_iterations: int = 1000,
    max_sideways: int = 100,
    max_restarts: int = 10,
    objective_name: str | None = None,
    run_index: int = 1,
) -> SearchResult[State]:
    """Run␣one␣hill-climbing␣variant␣and␣return␣a␣shared-contract␣SearchResult."""
    if variant not in VARIANTS:
        raise ValueError(f"variant␣must␣be␣one␣of␣{VARIANTS}")
    if type(seed) is not int:
        raise ValueError("seed␣must␣be␣an␣integer")
    _check_int(neighbors_per_iteration, "neighbors_per_iteration", 1)
    _check_int(max_iterations, "max_iterations", 0)
    _check_int(max_sideways, "max_sideways", 0)
    _check_int(max_restarts, "max_restarts", 0)
    _check_int(run_index, "run_index", 1)

    started_at = perf_counter()
    rng = Random(seed)
    objective = resolve_objective(initial_state, objective_fn)
    climb_kwargs = dict(
        variant="steepest" if variant == "random_restart" else variant,
        neighbors_per_iteration=neighbors_per_iteration,
        max_iterations=max_iterations,
        max_sideways=max_sideways,
    )

    if variant == "random_restart":
```

```
        # Restart 0 uses the supplied initial state; each later one a fresh random state.
        best_state, best_score = initial_state, None
        history: list[ObjectiveValue] = []
        per_restart: list[int] = []
        for restart in range(max_restarts + 1):
            start = (
                initial_state
                if restart == 0
                else random_initial_state(initial_state.packages, initial_state.truck, rng)
            )
            state, score, hist, iters, _ = _climb(start, objective, rng, **climb_kwargs)
            history.extend(hist)
            per_restart.append(iters)
            if best_score is None or score > best_score:
                best_state, best_score = state, score
        final_state, final_score = best_state, best_score
        iterations, sideways = sum(per_restart), 0
        restarts = max_restarts
    else:
        final_state, final_score, history, iterations, sideways = _climb(
            initial_state, objective, rng, **climb_kwargs
        )
        per_restart, restarts = [], 0

    parameters: dict[str, int | float | str | bool] = {
        "variant": variant,
        "max_iterations": max_iterations,
    }
    if variant != "stochastic":
        parameters["neighbors_per_iteration"] = neighbors_per_iteration
    if variant == "sideways":
        parameters["max_sideways"] = max_sideways
    if variant == "random_restart":
        parameters["max_restarts"] = max_restarts

    return SearchResult(
        algorithm=f"hill_climbing_{variant}",
        objective_name=objective_name or ("total_value" if objective_fn is None else "custom"),
        initial_state=initial_state,
        final_state=final_state,
        objective_value=final_score,
        objective_history=tuple(history),
        duration_seconds=perf_counter() - started_at,
        metrics=HillClimbingMetrics(
            variant=variant,
            iterations=iterations,
            sideways_moves=sideways,
            maximum_sideways_moves=max_sideways if variant == "sideways" else None,
            restarts=restarts,
            maximum_restarts=max_restarts if variant == "random_restart" else None,
            iterations_per_restart=tuple(per_restart),
        ),
        seed=seed,
        run_index=run_index,
        parameters=parameters,
    )
```

Listing 3.3: Fungsi pembungkus tiap varian pada `algorithm.py`

```
def steepest_ascent_hill_climbing(initial_state: State, **kw) -> SearchResult[State]:
    return hill_climbing(initial_state, variant="steepest", **kw)


def sideways_hill_climbing(initial_state: State, **kw) -> SearchResult[State]:
    return hill_climbing(initial_state, variant="sideways", **kw)


def stochastic_hill_climbing(initial_state: State, **kw) -> SearchResult[State]:
    return hill_climbing(initial_state, variant="stochastic", **kw)


def random_restart_hill_climbing(initial_state: State, **kw) -> SearchResult[State]:
    return hill_climbing(initial_state, variant="random_restart", **kw)
```

3.6.5 Pelaksana Eksperimen dan Visualisasi

Berkas `src/experiments/hill_climbing.py` menjalankan setiap varian sebanyak tiga kali dengan tiga *seed* berbeda. *State* awal setiap *run* dibangkitkan secara acak dari *seed* tersebut. Untuk setiap *run*, program menyimpan berkas `result.json`, plot nilai *objective function* terhadap iterasi, dan visualisasi tiga dimensi *state* awal dan akhir. Ringkasan lintas *run* (rata-rata dan simpangan baku nilai akhir, durasi, dan iterasi) disimpan pada `summary.json` dan `summary.png`. Program dijalankan melalui `src/experiments/run_hill_climbing.py`.

3.7 Implementasi Simullated Annealing

3.8 Implementasi Genetic Algorithm

Bab 4

Hasil Eksperimen dan Analisis

4.1 Metodologi Eksperimen

4.2 Hasil Eksperimen dan Analisis tiap Algoritma

4.2.1 Hill Climbing

4.3 Hasil Eksperimen dan Analisis tiap Algoritma

4.3.1 Hill Climbing

4.3.2 Simulated Annealing

4.3.3 Genetic Algorithm

4.4 Analisis Keseluruhan

Bab 5

Kesimpulan dan Saran

5.1 Kesimpulan

5.2 Saran

Bibliografi

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.

[2] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[3] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.

[4] D. L. Poole and A. K. Mackworth, *Artificial Intelligence: Foundations of Computational Agents*, 2nd ed. Cambridge University Press, 2017. [Online]. Available: https://artint.info/2e/html2e/ArtInt2e.Ch4.S7.SS1.html